

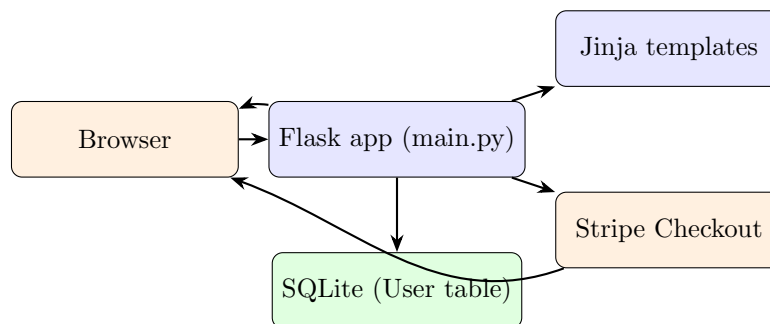
Flask Online Shop — Brief

Explainer document generated from the project source

What it is

A single-file Flask web app: a nine-item “planet” product catalog, account registration/login backed by SQLite, and a “Buy” flow that hands off to Stripe’s hosted Checkout page. No cart, no JavaScript framework, no build step — Flask renders HTML templates server-side and the browser submits plain HTML forms.

Architecture at a glance



Four responsibilities, all in `main.py`: catalog data (two dictionaries merged into display rows), a one-table user database, auth (register/login/logout), and payment (one route that creates a Stripe Checkout Session and redirects to it).

The catalog and the price-consistency guarantee

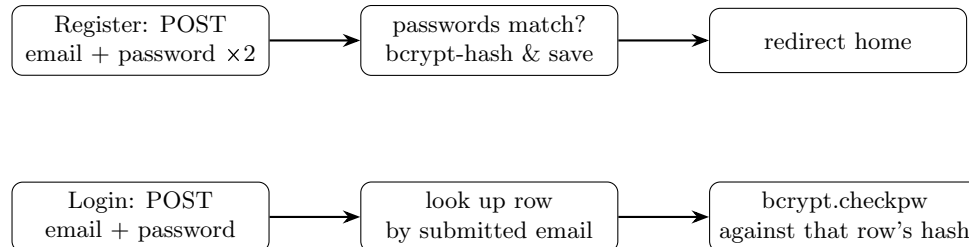
`planet_price_dict` holds the exact shape Stripe’s Checkout API needs (currency, product name, `unit_amount` in cents). `PLANET_INFO` holds display copy (name, image, description). `get_planets()` joins the two and runs the price through `format_price()` so the number shown on a product card is *always derived from* the same number sent to Stripe — they cannot drift apart. This mattered concretely: the home page used to hardcode “\$69,420” for Uranus while the real Stripe amount for Uranus was \$42,069; every other planet’s hardcoded price happened to agree by coincidence. The fix removes the second, hand-typed copy entirely.

Data model

One table, `User(id, email, password, logged_in)`. Passwords are hashed with bcrypt (a slow, salted hashing algorithm built for passwords) — never stored or compared in plaintext. `logged_in`

is an integer flag on the row itself, not a per-browser session value: `is_logged_in()` asks “does any row have `logged_in = 1`?” and a Flask **context processor** injects that answer into every template’s nav bar. This is simple but means login state is global across browsers, not isolated per session — a known, stated limitation, not a hidden bug.

Auth flows



A real bug fixed here: the original login code looked up whether the submitted email existed, but then always compared the submitted password against *user id 1*’s stored hash, regardless of which email was submitted. With exactly one account this is invisible; with two accounts it lets a wrong password through for user 2 (if it matches user 1’s hash) or rejects user 2’s own correct password. The fix looks up and checks against the row matching the *submitted* email. This is a good example of a bug that survives solo testing and only shows up with a second real user.

Checkout flow

Clicking “Buy <planet>” POSTs the planet’s slug to `/create-checkout-session`, which looks up that planet’s Stripe line item and calls `stripe.checkout.Session.create(...)`, then issues a **303 redirect** (tells the browser to re-fetch the target with GET) to Stripe’s own hosted payment page. The user pays entirely on Stripe’s infrastructure, then Stripe redirects back to `/success` or `/cancel`. There is no cart — each click is its own single-item Checkout Session.

What was verified, and what was not (stated plainly)

- **Verified:** the route was hit directly with a valid planet name and no Stripe key configured; it reached `stripe.checkout.Session.create` and failed only at Stripe’s own “no API key” check — confirming the route is wired correctly up to the point a real key is required.
- **Not verified:** a full, real Stripe payment end to end (creating a live test session, completing payment, landing on `/success` as a result of that payment). No real Stripe test-mode secret key was ever available in this project’s environment, so this path has never actually been exercised.
- There is no webhook confirming payment server-side, so `/success` can be visited directly without paying — a known, stated limitation.

Templates and styling

All pages extend one `base.html` (nav, flashed messages, footer, Bootstrap/Stripe script tags), overriding only a `content` block. A CSS custom-property system (colors, radius, shadow defined once under `:root`) drives a small set of reusable component classes — product cards, an auth-card

shell shared by login/register, and a result-card shell shared by success/cancel — so all five page types share one visual language despite looking different.

Configuration and known gaps

The Flask session-signing key and the Stripe secret key are both read from environment variables (documented in `.env.example`), replacing an earlier version where both were hardcoded directly in source — a real security fix, not a stylistic one. Stated, not-yet-fixed limitations carried over from the project’s own README:

- No CSRF token on any form.
- Password length is capped only by an HTML `maxlength` attribute, not enforced server-side.
- No server-side email format validation.
- Login state is a global per-row database flag, not a per-session cookie value.
- No Stripe webhook to confirm a payment actually happened before showing `/success`.

Glossary

Route	A URL path + HTTP method mapped to a handler function.
ORM	A library (here, Flask-SQLAlchemy/SQLAlchemy) that represents database rows as Python objects instead of raw SQL.
Session/cookie	A signed piece of browser-stored state Flask uses to remember things (like a one-time flash message) between requests from the same browser.
Hashing	A one-way transform (bcrypt here) used to store passwords without ever storing the plaintext.
CSRF	An attack tricking a logged-in user’s browser into submitting an unwanted request; normally defended against with a per-session form token, which this app does not have.
303 redirect	An HTTP status telling the browser to re-fetch the target URL with <code>GET</code> , regardless of the original request’s method.
Context processor	A Flask hook that injects extra variables into every template’s rendering context automatically.

Flagged ambiguities

No automated test suite exists; all verification claims above are carried over from the project’s own README account of manual testing, not independently re-run while writing this document. The full Stripe payment round trip remains genuinely unverified for the stated reason (no real Stripe key available) — this is not a documentation gap, it is an accurate reflection of the project’s real, disclosed state.